



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

“EcoFleet AI: Intelligent Electric Vehicle Fleet Monitoring and Route Optimization Platform”

ARCHITECTURE DESIGN REPORT

Submitted in the partial fulfilment for the Course of

CSA1012-SOFTWARE ENGINEERING

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE(AI)

Submitted by

P L OAVIKSHA (192572001)

Under the Supervision of

Dr.ANITA DAVAMANI K

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

AUGUST 2026

Table of Contents	Page No.
Abstract	1
Introduction	1
1. Industry Context and Problem Statement	2
2. System Requirements Analysis	3
2.1 Functional Requirements	3
2.2 Non-Functional Requirements & Quality Attributes	4
3. Selection of Software Architecture	5
4. Architecture Diagram – Overall System Structure	6
5. Data Flow Diagram	7
6. Deployment Architecture	8
7. Workflow Diagram – Predictive Maintenance and Charging	9
8. Quality Attribute Analysis	10
9. Architectural Justification and Trade-offs	11
Conclusion	12
References	13

EcoFleet AI – Architecture Design Report

Intelligent Electric Vehicle Fleet Monitoring and Route Optimization Platform

Abstract

EcoFleet AI is a proposed intelligent platform for monitoring electric vehicle (EV) fleets and optimizing their charging and routing decisions. Conventional fleet management tools track vehicle location but offer little insight into battery health, charging efficiency, or route-level energy consumption — gaps that grow more costly as fleets scale. This report analyzes the functional and non-functional requirements of an AI-driven EV fleet platform and proposes a hybrid microservices and event-driven architecture built around an IoT edge-ingestion layer, a Kafka-based streaming backbone, independently deployable prediction and optimization services, and a secured API gateway serving fleet manager and driver applications. The design is evaluated against scalability, reliability, availability, performance, security, and maintainability, and is illustrated through four complementary views: an overall system architecture diagram, a Level-1 data flow diagram, a deployment architecture diagram, and a predictive-maintenance workflow diagram. The report concludes that this architecture directly supports EcoFleet AI's central requirement continuous, multi-concern intelligence over a growing EV fleet while remaining adaptable to future operational and regulatory needs.

Introduction

The global shift toward electric vehicles is reshaping fleet operations across logistics, public transport, and last-mile delivery. Operators adopting EVs gain lower emissions and fuel costs, but take on a new operational burden: batteries degrade in ways that are hard to observe directly, charging must be scheduled around tariffs and station availability, and route planning now has to account for vehicle range and energy consumption rather than distance alone. Existing fleet management systems were largely built for internal-combustion vehicles and, at best, bolt on basic GPS tracking for EVs — they do not predict battery health, optimize charging, or factor energy efficiency into routing decisions.

This report addresses that gap by designing the software architecture for EcoFleet AI, a platform intended to give fleet operators continuous, predictive visibility into their EV fleets. The report follows a structured architecture-design process: it begins by analyzing the system's functional and non-functional requirements and the quality attributes that should drive the

design (Section 2); selects and justifies an appropriate architectural style (Section 3); presents the resulting architecture through four diagrams covering the overall system, data flow, deployment, and a representative operational workflow (Sections 4–7); evaluates the design against key quality attributes (Section 8); and closes with a discussion of architectural trade-offs and directions for future enhancement (Section 9). The goal throughout is not just to describe a working system, but to justify why this particular architecture — rather than the alternatives considered — is the right fit for the demands of AI-driven EV fleet management.

1. Industry Context and Problem Statement

The transportation industry is transitioning rapidly toward electric vehicles (EVs) to cut carbon emissions, reduce fuel dependency, and support sustainable mobility. Logistics, public transport, and delivery fleets are adopting EVs at scale to improve efficiency and meet environmental regulations. Managing an EV fleet, however, requires continuous monitoring of battery health, charging schedules, energy consumption, and route efficiency — a task that goes well beyond simple vehicle tracking.

Conventional EV fleet management systems focus on location tracking but lack the intelligence to predict battery degradation, optimize charging schedules, or minimize energy consumption. Poor route planning results in unnecessary charging stops, higher operating costs, and delivery delays, while fleet managers often lack real-time visibility into vehicle health and charging station availability. EcoFleet AI addresses this by continuously monitoring fleet performance, predicting maintenance needs, optimizing charging operations, and recommending energy-efficient routes.

Objectives: develop an AI-powered platform that monitors EV fleet performance end-to-end; continuously track battery health, energy consumption, and vehicle status via IoT sensors; predict battery degradation and maintenance needs with ML; optimize charging schedules against tariffs and charger availability; recommend energy-efficient routes; and provide real-time dashboards and analytical reporting.

Expected outcomes: longer battery life, higher fleet utilization, lower energy/charging costs, shorter travel times, fewer unplanned outages, and a scalable platform supporting sustainable transportation.

2. System Requirements Analysis

2.1 Functional Requirements

- Ingest real-time telemetry (state-of-charge, temperature, voltage, current) from IoT sensors on every vehicle
- Predict battery degradation and upcoming maintenance needs using ML models trained on historical sensor data
- Recommend and schedule optimal charging windows based on battery status, tariffs, and charger availability
- Compute energy-efficient routes using live GPS, traffic, and vehicle load data
- Provide a real-time fleet dashboard (vehicle status, battery health, routes, charging queue)
- Generate periodic analytical reports on utilization, energy efficiency, and cost savings
- Send alerts for critical battery conditions, missed maintenance windows, or charging failures

2.2 Non-Functional Requirements & Quality Attributes

Attribute	Requirement
Scalability	Support a growing fleet — hundreds to tens of thousands of vehicles — and rising telemetry volume without redesign
Reliability	Predictive maintenance and routing outputs must stay accurate and available even under partial component failure
Availability	Dashboards and charging recommendations need near-continuous uptime, since fleets run around the clock
Performance	Telemetry ingestion and route recommendations require low-latency, near real-time processing
Security	Vehicle data, location history, and charging infrastructure access must be protected from tampering and unauthorized access
Maintainability	ML models, routing logic, and charging rules will change often and must be independently updatable

3. Selection of Software Architecture

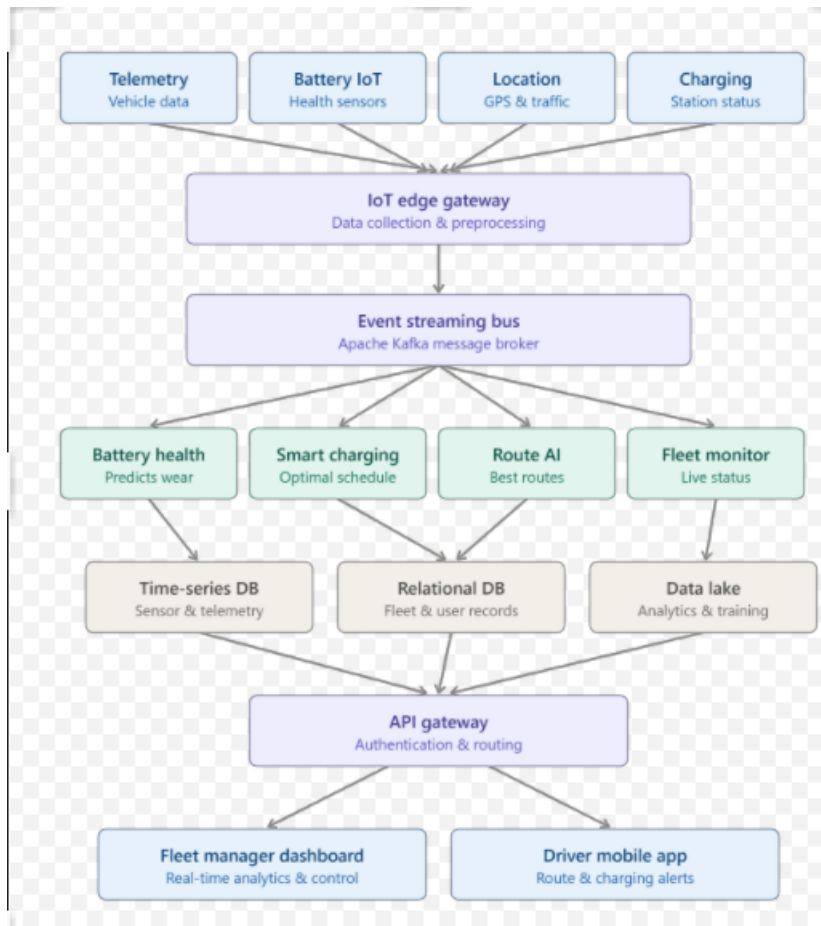
Selected style: Hybrid Microservices + Event-Driven Architecture, layered on an IoT edge-ingestion tier, with a cloud API gateway and client layer.

Justification:

- Microservices let battery-health prediction, charging optimization, route optimization, and fleet monitoring evolve, scale, and fail independently — each has very different compute needs (ML inference vs. optimization algorithms vs. simple dashboards).
- Event-driven communication via a streaming message bus is the natural fit for continuous IoT telemetry: thousands of vehicles emit readings asynchronously, and multiple services need to consume the same stream without tight coupling.
- A monolithic or purely layered design would bottleneck as the fleet grows and would make it hard to update ML models independently of routing logic.
- A pure serverless design was considered but rejected as the *primary* style — continuous telemetry streams and stateful ML inference are less cost-efficient on pure function-as-a-service. Select services (e.g. periodic reporting) do use serverless functions, which is what makes the design genuinely hybrid rather than a single pattern applied everywhere.

4. Architecture Diagram Overall System Structure

The diagram below shows the full structure: IoT sources flow through an edge gateway into an event streaming bus, which fans data out to independent microservices; each writes to the data layer it needs, and all client access is mediated through one API gateway.



Components: IoT sensors stream via the edge gateway into Kafka, which fans out to four independent microservices (battery health prediction, charging optimization, route optimization, fleet monitoring). Each writes to the store it needs — time-series for sensor data, relational for fleet/user records, a data lake for ML training. The API gateway is the single secured entry point for both client apps.

5. Data Flow Diagram

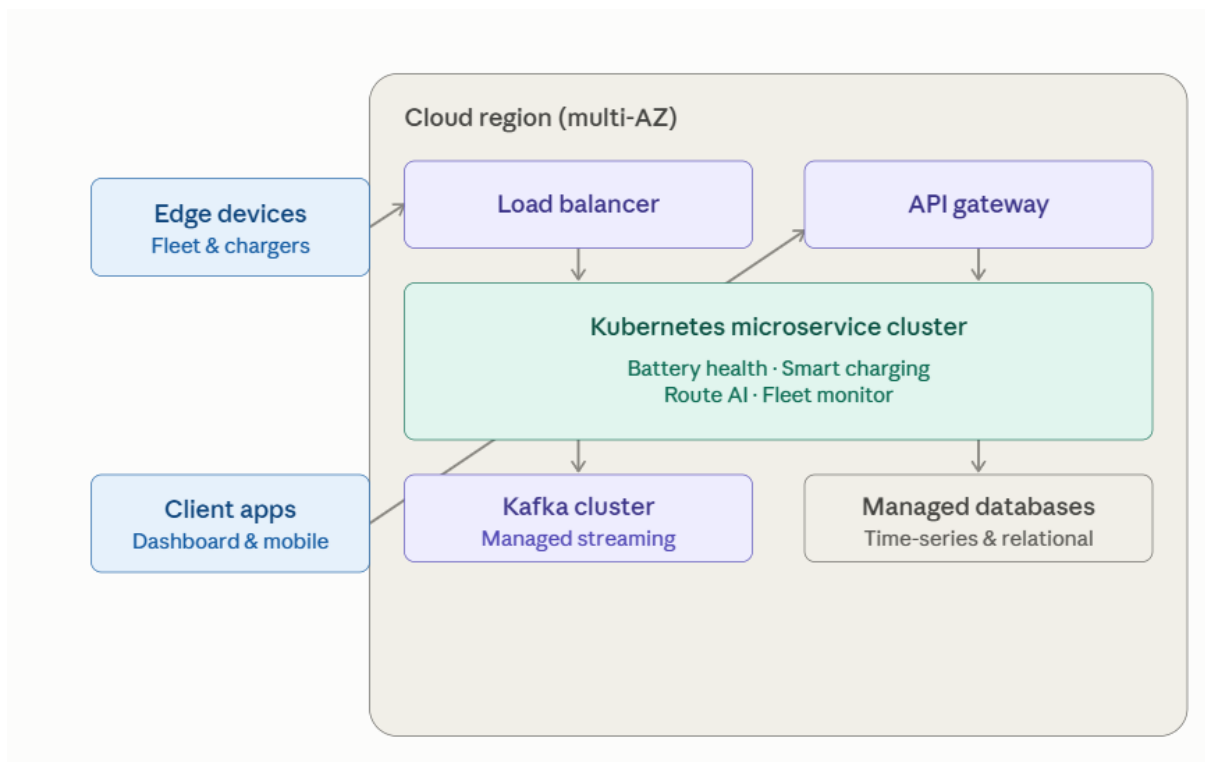
The DFD traces information between external entities, processes, and data stores — useful for confirming every requirement in Section 2 has a data path.



Vehicles and charging stations feed process 1.0/2.0, which drives both 3.0 (charging) and 4.0 (route optimization) — both reading and writing the time-series and relational stores. Process 5.0 consolidates the outputs and delivers them to the fleet manager and driver.

6. Deployment Architecture

The deployment view maps the logical architecture onto physical/cloud infrastructure.

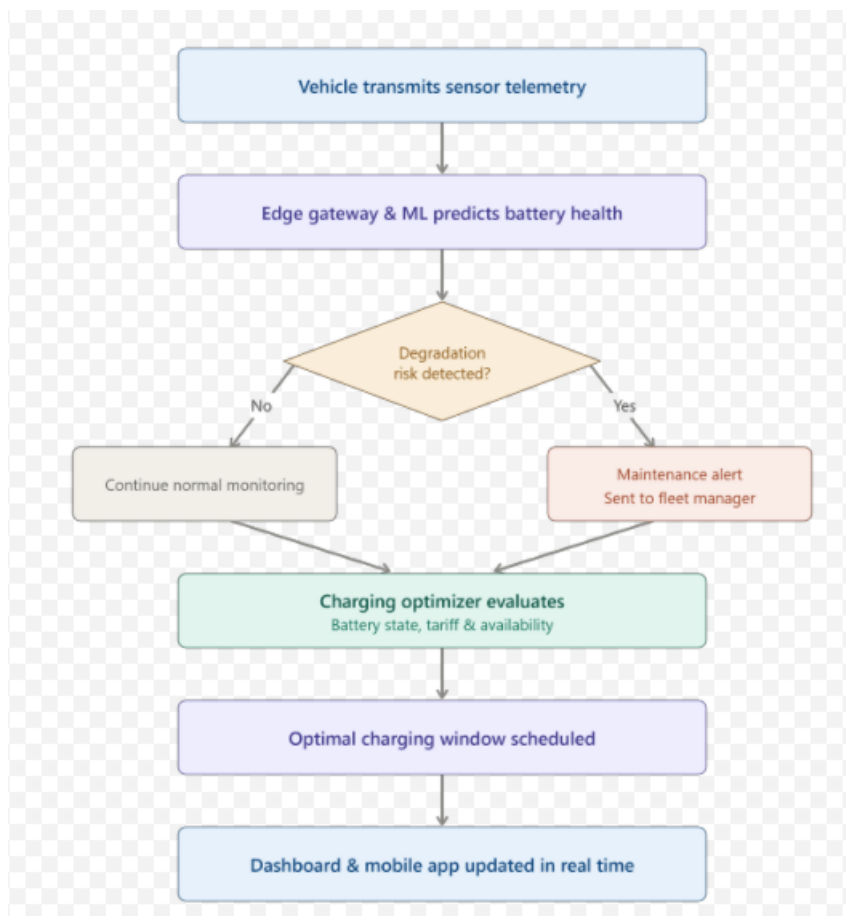


EV telematics and charging-station gateways connect over cellular/IoT networks to a load balancer at the region edge. A Kubernetes cluster hosts each microservice as an independently scalable pod group, alongside a managed Kafka cluster and managed databases (time-series,

relational, and — not pictured here for space — a data lake) in the same region for low-latency access. Clients never connect directly to internal services; everything routes through the load balancer and API gateway.

7. Workflow Diagram — Predictive Maintenance and Charging

This traces one core scenario end-to-end: how a telemetry reading results in either a maintenance alert or an optimized charging schedule.



Telemetry flows from the vehicle through the edge gateway to the ML-based battery health model. If degradation risk is detected, a maintenance alert reaches the fleet manager; otherwise monitoring continues normally. Either way, the charging optimizer evaluates battery state, tariff, and availability, schedules the optimal window, and pushes the update to the dashboard and mobile app in real time.

8. Quality Attribute Analysis

Attribute	How the architecture addresses it
Scalability	Each microservice scales independently; Kafka partitioning lets telemetry throughput grow with fleet size without redesign
Security	The API gateway centralizes authentication/authorization (OAuth2/JWT); data in transit is encrypted, and sensitive data is access-controlled per role
Performance	Event-driven ingestion avoids synchronous bottlenecks; the time-series database is optimized for fast writes/reads of high-frequency sensor data
Reliability	Kafka retains events, so a temporarily-down consumer doesn't lose data; services fail independently rather than taking the whole platform down
Maintainability	Loosely coupled microservices mean the battery ML model or routing algorithm can be redeployed without touching other services
Availability	Stateless microservices behind the API gateway run multiple replicas; the message bus decouples producers and consumers so a slow consumer never blocks ingestion

9. Architectural Justification and Trade-offs

Rationale: EV fleet management is fundamentally a continuous-stream, multi-concern problem battery science, energy pricing, live routing, and operational monitoring all move at different speeds and evolve independently. A hybrid microservices + event-driven design maps directly onto that: the event bus handles the "many sensors, many consumers" nature of telemetry, while microservices let each concern be owned, scaled, and evolved on its own schedule.

Trade-offs considered:

- *Complexity vs. flexibility* microservices add operational overhead (deployment, monitoring, contracts) vs. a monolith, justified by how differently each service needs to scale.
- *Eventual vs. strict consistency* a dashboard metric may lag the source event by seconds, acceptable for fleet monitoring but worth stating explicitly.

- *Serverless vs. always-on* bursty tasks (periodic reporting) suit serverless; latency-sensitive, stateful services (ML inference, routing) remain persistently-running microservices hence *hybrid*.

Conclusion

This report has walked through the full architecture design process for EcoFleet AI, from requirements analysis to a justified, diagrammed architectural proposal. The core finding is that EV fleet management is not a single problem but several loosely related ones running at different speeds real-time telemetry ingestion, predictive battery modeling, charging optimization, and route planning and that a hybrid microservices and event-driven architecture is well suited to exactly that shape of problem. By routing all telemetry through a central event streaming bus and letting each concern live in its own independently deployable service, the design allows the platform to scale with fleet size, tolerate partial failures without full outages, and evolve its ML models and optimization logic without disrupting the rest of the system.

The four architectural views presented system architecture, data flow, deployment, and workflow

together give a complete picture of how the platform is structured, how information moves through it, how it would actually be hosted, and how a single operational scenario plays out end to end. Evaluated against the quality attributes identified at the outset, the architecture holds up well on scalability, reliability, and maintainability in particular, at the cost of the operational complexity inherent to any microservices system and a deliberate acceptance of eventual consistency in dashboard data. These are reasonable trade-offs for the problem at hand. As the platform matures, its event-driven backbone means new capabilities — driver scoring, carbon-credit reporting, additional charging-network integrations — can be added as new subscribers to the existing data stream, positioning EcoFleet AI to grow alongside the fleets it serves.



References

1. Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4th ed.). Addison-Wesley.
2. Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.
3. Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.
4. Fowler, M., & Lewis, J. (2014). *Microservices*. martinfowler.com.
5. Kreps, J., Narkhede, N., & Rao, J. (2011). *Kafka: A Distributed Messaging System for Log Processing*. Proceedings of the NetDB Workshop.